St. Anne's Arts and Science College

Department of Computer Science

Team Members	Register No	Project Name	Phase Topic
JAISREE.R	222409465	FitTrack AI - Personalized Fitness Recommendations Powered by AI	Phase 4 PROJECT PLANNING
JANU SRI.D	222409466		
JECHINTHA JOYCE.S	222409467		
JENITA.G	222409468		

FITTRACK AI

Personalized Fitness Recommendations Powered by AI

PHASE 4

PROJECT PLANNING PHASE

Project Name: AI FitTrack

Technology Stack: Node.js, Express.js, MongoDB, Mongoose, JWT, bcrypt.js, Google Gemini AI

CONTENTS

Section	Topic
4.1	Project Planning Overview
4.2	Project Structure
4.3	Development Environment Setup
4.4	Environment Configuration
4.5	Module Planning
4.5.1	Server Configuration
4.5.2	Database Configuration
4.5.3	Authentication Module
4.5.4	Workout Management Module
4.5.5	AI Recommendation Module
4.5.6	Middleware and Error Handling
4.6	Planned Development Sequence
4.7	Planned API Testing Arrangement
4.8	Project Development Schedule
4.9	Phase Summary

PHASE 4 – PROJECT PLANNING PHASE

4.1 Project Planning Overview

Following the requirement analysis and system design activities, the development of AI FitTrack was organized into a structured plan. The work was divided into manageable modules to support the orderly development and integration of authentication, workout management, database operations, and AI-based services.

The project is backend-oriented and is organized into separate client and server areas. The server contains the principal application logic and exposes the RESTful APIs used by clients and API testing tools.

4.2 Project Structure

The planned project directory separates the client area from the backend server. This arrangement provides a clear location for the API implementation and for applications or tools that consume the APIs.

```
AI-FitTrack-API/\n|\n├── client/\n|\n└── server/
```

The server directory is intended for backend implementation, while the client directory represents the area from which the APIs can be consumed.

Figure 4.1: Planned Project Directory Structure

[INSERT PROJECT FOLDER STRUCTURE SCREENSHOT HERE]

4.3 Development Environment Setup

The development environment was planned around Node.js and npm for backend development. Visual Studio Code was selected as the code editor, and MongoDB was selected as the database system.

The initial server setup consists of the following activities:

1. Create the main project directory.
2. Open the project directory in Visual Studio Code.
3. Create the server directory for backend files.
4. Initialize the Node.js project using the npm init -y command.
5. Install the packages required for the backend and add Nodemon as a development dependency.

The planned application dependencies are listed below.

Package	Planned purpose
express	Create the backend server and define REST API routes.
mongoose	Define data models and interact with MongoDB.
dotenv	Load configuration values from environment variables.
bcryptjs	Protect user passwords through hashing.
jsonwebtoken	Create and verify JSON Web Tokens for authentication.
cors	Configure Cross-Origin Resource Sharing for API access.
nodemon	Restart the server automatically during development; used as a development dependency.

4.4 Environment Configuration

Environment variables were planned to keep configuration values separate from the application source code. The planned configuration includes the server port, MongoDB connection string, JWT secret, and Google Gemini API key.

```
PORT=5000\nMONGO_URI=mongodb://localhost:27017/aifittrack\nJWT_SECRET=your_secret_key\nG
EMINI_API_KEY=your_google_gemini_api_key
```

These values serve the following purposes:

- PORT specifies the port on which the backend server is configured to listen.
- MONGO_URI provides the connection string used to connect to the MongoDB database.
- JWT_SECRET is used in the JWT authentication process and must be kept private.
- GEMINI_API_KEY provides the credential required to access the Google Gemini service.

Actual secret values should be kept out of source code and should not be included in submitted screenshots or public repositories.

4.5 Module Planning

Development was divided into the following modules. Each module has a defined responsibility and contributes to the overall backend system.

4.5.1 Server Configuration

Establishes the Express server, loads environment variables, configures JSON parsing and CORS, and prepares the application to receive API requests.

4.5.2 Database Configuration

Establishes the connection between the backend application and MongoDB using Mongoose.

4.5.3 Authentication Module

Handles user registration and login. Passwords are protected using bcrypt.js, and JWT is used to authenticate access to protected resources.

4.5.4 Workout Management Module

Supports the creation, retrieval, updating, deletion, and searching of workout records.

4.5.5 AI Recommendation Module

Connects the backend with Google Gemini to provide personalized workout recommendations and fitness insights.

4.5.6 Middleware and Error Handling

Provides middleware for authentication and request protection. Centralized error handling is planned to manage API errors consistently.

4.6 Planned Development Sequence

The development activities were arranged in a sequence that moves from the initial backend setup to implementation, integration, and testing.

1. Set up the Node.js backend project.
2. Configure the Express server.
3. Establish the MongoDB connection.
4. Create the User and Workout models.
5. Implement user registration and login.
6. Implement JWT-based authentication and protected routes.
7. Develop workout management APIs.
8. Add workout search functionality.
9. Integrate Google Gemini AI services.
10. Implement personalized recommendations and fitness insights.
11. Add validation and centralized error handling.
12. Test the completed APIs using an API testing tool.

This sequence provides a structured progression from the basic backend environment to the complete AI-assisted fitness service.

4.7 Planned API Testing Arrangement

API testing was planned after implementation of the major backend modules. Testing activities are grouped according to the main API areas.

API group	Planned testing coverage
Authentication APIs	User registration, login, and profile access.
Workout APIs	Adding, retrieving, updating, and deleting workout records; workout search.
AI APIs	Workout recommendations and fitness insights.

The testing arrangement is intended to verify the behavior of each API group after the corresponding functionality has been implemented.

4.8 Project Development Schedule

The overall work is organized into the following development stages. The stages represent the planned order of activities; specific calendar dates and durations have not been assigned in this plan.

Stage	Planned work
Stage 1	Project and environment setup
Stage 2	Database configuration and models
Stage 3	User authentication
Stage 4	Workout management
Stage 5	AI service integration
Stage 6	Middleware and error handling
Stage 7	API testing
Stage 8	Final demonstration

4.9 Phase Summary

The planning phase organizes the AI FitTrack development work into a sequence of manageable activities and clearly defined modules. It identifies the project structure, development environment, configuration

requirements, implementation sequence, API testing arrangement, and major development stages. This plan provides a structured basis for implementing the backend and preparing it for testing and demonstration.